MLOps + LangChain Portfolio Project: Intelligent Document Analysis System

Project Overview

Build an **end-to-end intelligent document analysis system** that combines RAG (Retrieval-Augmented Generation) with production MLOps practices. This system will analyze business documents, extract insights, and provide intelligent Q&A capabilities.

Why This Project Works

- **Real business value**: Document analysis is a common enterprise need
- **Shows MLOps skills**: Full ML lifecycle with monitoring and deployment
- **Demonstrates LangChain expertise**: Advanced RAG implementation
- **Production-ready**: Not just a demo, but deployable solution
- **Scalable architecture**: Shows you understand enterprise requirements

Tech Stack

- **LangChain/LangGraph**: Document processing and RAG implementation
- **MLOps**: MLflow, DVC, Docker, CI/CD
- **Vector DB**: Pinecone/ChromaDB for embeddings
- **Monitoring**: Weights & Biases, Prometheus
- **Deployment**: FastAPI, Docker, AWS/GCP
- **Frontend**: Streamlit for demo interface

Project Structure

```
intelligent-doc-analyzer/
├── src/
│   ├── data_pipeline/        # Document ingestion & preprocessing
│   ├── embedding_service/    # Vector embeddings generation
│   ├── rag_engine/           # LangChain RAG implementation
│   ├── api/                  # FastAPI service
│   └── monitoring/           # MLOps monitoring
├── models/                   # Model artifacts & configs
├── data/                     # Sample documents
├── tests/                    # Unit & integration tests
├── deployment/               # Docker, K8s configs
├── experiments/              # MLflow experiments
└── notebooks/                # Analysis & exploration
```

Core Features to Implement

1. Document Processing Pipeline (MLOps Focus)

- **Data Versioning**: Use DVC for document version control
- **Pipeline Orchestration**: MLflow pipelines for data processing
- **Quality Monitoring**: Document ingestion success rates, processing times
- **Automated Testing**: Data validation and pipeline testing

2. RAG Engine (LangChain Focus)

- **Multi-format Support**: PDF, Word, Excel, PowerPoint
- **Advanced Chunking**: Semantic chunking with LangChain
- **Multi-modal RAG**: Handle text, tables, and images
- **Query Planning**: Use LangGraph for complex query routing
- **Response Validation**: Fact-checking and source attribution

3. Production Deployment (MLOps)

- **API Service**: FastAPI with async endpoints
- **Model Serving**: Containerized deployment
- **Auto-scaling**: Based on request volume
- **Health Checks**: Service and model health monitoring
- **A/B Testing**: Compare different RAG configurations

4. Monitoring & Observability

- **Performance Metrics**: Response time, accuracy, user satisfaction
- **Model Drift**: Embedding drift detection
- **Business Metrics**: Query patterns, popular documents
- **Alerting**: Automated alerts for failures or performance degradation

Implementation Phases

Phase 1: Core RAG System (Week 1)

```
python
```

```
# Key components to build:
1. Document loader (LangChain)
2. Embedding generation and storage
3. Basic Q&A functionality
4. Simple web interface
```

Phase 2: MLOps Integration (Week 2)

```python
# Add production elements:
1. MLflow experiment tracking
2. Model versioning and registry
3. Automated testing pipeline
4. Docker containerization
```

Phase 3: Advanced Features (Week 3)

```python
# Production-ready features:
1. LangGraph for complex queries
2. Monitoring dashboard
3. API rate limiting and auth
4. Performance optimization
```

Phase 4: Deployment & Monitoring (Week 4)

```python
# Full production deployment:
1. Cloud deployment (AWS/GCP)
2. CI/CD pipeline
3. Monitoring and alerting
4. Documentation and demos
```

Sample Code Structure

Main RAG Engine

```python
```

```python
# src/rag_engine/rag_system.py
from langchain.chains import RetrievalQA
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Pinecone
import mlflow

class ProductionRAGSystem:
    def __init__(self, config):
        self.config = config
        self.setup_mlflow_tracking()
        self.load_models()

    def setup_mlflow_tracking(self):
        mlflow.set_tracking_uri(self.config.mlflow_uri)
        mlflow.set_experiment("document-rag-system")

    def process_query(self, query, session_id):
        with mlflow.start_run():
            # Log query and track performance
            mlflow.log_param("query_length", len(query))

            response = self.rag_chain.invoke(query)

            # Log metrics
            mlflow.log_metric("response_time", response.metadata.get("time"))
            mlflow.log_metric("confidence_score", response.metadata.get("confidence"))

            return response
```

MLOps Pipeline

python

```
# src/mlops/pipeline.py
from mlflow.pipelines import Pipeline
import dvc.api

class DocumentProcessingPipeline:
    def __init__(self):
        self.setup_dvc_tracking()

    def run_pipeline(self):
        # Data ingestion with versioning
        # Model training/fine-tuning
        # Model validation
        # Model registration
        # Deployment trigger
        pass
```

Key Metrics to Track

Technical Metrics

- **Response Time**: < 2 seconds for 95% of queries
- **Accuracy**: Measured via human evaluation
- **Embedding Quality**: Cosine similarity scores
- **System Uptime**: 99.9% availability

Business Metrics

- **User Engagement**: Query frequency, session duration
- **Document Coverage**: % of documents being queried
- **Cost per Query**: Infrastructure and API costs

Deployment Strategy

Local Development

```
bash

# Docker compose for local development
docker-compose up -d  # Starts all services locally
```

Production Deployment

```
bash
```

```
# Kubernetes deployment
kubectl apply -f deployment/k8s/
```

CI/CD Pipeline

- **GitHub Actions**: Automated testing and deployment
- **Model Registry**: MLflow model promotion
- **Gradual Rollout**: Blue-green deployments

Success Metrics for Your Portfolio

Technical Excellence

- **Code Quality**: Clean, tested, documented code
- **Production Patterns**: Proper error handling, logging, monitoring
- **Scalability**: Can handle increasing load

MLOps Maturity

- **Experiment Tracking**: All experiments logged and reproducible
- **Model Versioning**: Clear model lineage and rollback capability
- **Monitoring**: Comprehensive observability

Business Value

- **Real Use Case**: Solves actual business problems
- **Performance**: Fast, accurate, reliable
- **User Experience**: Intuitive interface and clear responses

Portfolio Presentation Tips

GitHub Repository

- **Professional README**: Clear setup instructions, architecture diagrams
- **Live Demo**: Deployed version with sample data
- **Video Walkthrough**: 5-minute demo video

Technical Interview Prep

- **Architecture Decisions**: Be ready to explain design choices
- **Scaling Challenges**: Discuss how you'd handle 10x growth
- **Trade-offs**: Performance vs. cost vs. accuracy discussions

Resume Bullet Points

- "Built production RAG system processing 10k+ documents with 99.9% uptime"
- "Implemented MLOps pipeline reducing model deployment time by 80%"
- "Designed monitoring system tracking 15+ metrics for model performance"

This project demonstrates you can build production AI systems, not just prototypes. It shows both depth (MLOps practices) and breadth (LangChain expertise) that employers are looking for.